GCP Clinical Data Engineering Project

Part 1 - Project Overview and Architecture

Implementation-based reference | October 2026 | Synthetic clinical dataset

Evidence standard: observed implementation and logs are separated from unverified production claims.

Executive summary

Clinical Trial Data Ingestion and Medallion Architecture Pipeline | Healthcare / clinical studies. This is a hands-on GCP implementation using synthetic clinical datasets, not a verified production deployment for a real clinical client. It automates ingestion of 12 related clinical CSV feeds, distinguishes new/changed/unchanged rows, applies guarded incremental MERGE operations, and publishes curated data for study analytics.

60–90 second interview pitch

I built a clinical data engineering pipeline on Google Cloud using Cloud Run, Cloud Storage, Cloud Composer (Airflow), and BigQuery. A Cloud Run generator produces 12 related clinical CSV feeds into a batch-specific GCS landing path. Airflow validates files, loads staging tables, compares records to Bronze, performs a dry run, archives Bronze, and executes an incremental MERGE with audit records. A separate task transforms the 12 Bronze tables into typed, deduplicated Silver tables using BigQuery SQL, then Gold views and materialized views support study enrollment, treatment and event monitoring. The workflow supports read-only testing and explicitly confirmed live reruns.

Business objectives

Replace manual clinical feed loading with a repeatable, traceable pipeline; preserve raw source fidelity; avoid full reloads when incremental processing suffices; make study, patient, site and treatment information easier to report; and support recoverability via Bronze archives.

Evidence-based project metrics

The synthetic generator produced 12 CSV files and a previously observed 53,500 rows per batch. In one successfully executed live MERGE, comparison totals were 1,652 new, 3,858 changed and 49,595 unchanged records. Bronze increased from 55,105 to 56,757 rows; the pre-merge archive contained 55,105 rows. These are observed exercise figures, not daily SLA or client production metrics. No verified TB/day, cost-reduction percentage, uptime SLA or before/after latency benchmark is available.

End-to-end architecture

Cloud Run clinical-data-generator -> GCS gs://personalproject_1/clinical_source/{batch_id}/{table}.csv -> file validation -> BigQuery clinical staging (12 tables) -> incremental comparison -> dry-run verification -> Bronze archive -> Bronze MERGE (paraxel_bronze) -> Bronze-to-Silver MERGE (paraxel_silver) -> Gold reporting (paraxel_gold).

Airflow coordinates all steps. A separate existing Flask API branch loads data through its own Bronze/Silver path.

Platform inventory

Project: vaarahigcpproject; Composer environment: paraxelairflow; Composer region: us-central1; BigQuery location: US; DAG: clinical_data_pipeline; scheduled expression: 0 * * * *; max_active_runs: 1. Datasets: paraxel_bronze, paraxel_bronze_archive, paraxel_silver, paraxel_gold, clinical_control, plus staging datasets/tables.

Clinical entities

adverse_events; clinical_sites; clinical_studies; investigators; lab_results; medical_history; medications; patient_enrollments; patient_visits; patients; study_treatments; vital_signs. Primary keys are defined in the existing merge_clinical_incremental.KEYS mapping; do not assume key names without reading that mapping.

Why each technology

Cloud Run: managed container execution for generating batch feeds. GCS: durable, decoupled landing and archives. BigQuery: serverless warehouse with SQL MERGE, typed tables and reporting views. Composer/Airflow: task dependencies, retries, branches and GCP integration. Cloud Logging: operational task diagnostics. Standard views: always query current Silver data. Materialized views: precomputed results subject to BigQuery refresh rules.

Gold assets already created

Standard views: v_study_enrollment_summary and v_treatment_analysis. Materialized views: mv_daily_event_monitoring and mv_study_enrollment_status. Confirmed standard-view dependencies include Silver clinical_studies, patient_enrollments, clinical_sites and study_treatments. Materialized-view SQL definitions were not retrieved; do not claim their precise source expressions.

Scope and honesty

Pub/Sub, SFTP, source SQL databases, Dataflow, Spark execution, Unity Catalog, star-schema marts, CMEK and masking are possible alternatives or extensions, not verified components of this completed 12-table pipeline. An earlier separate Dataproc/PySpark practice build does not prove Spark was used in this specific Composer DAG.

Part 2 - Implementation and Troubleshooting

Implementation-based reference | October 2026 | Synthetic clinical dataset

Evidence standard: observed implementation and logs are separated from unverified production claims.

DAG behavior and operating modes

normal: Cloud Run generator plus the Flask branch, validation, staging, comparison, summary, dry-run, verified totals, Bronze archive, live Bronze MERGE, then 12-table Silver MERGE. test_existing_batch: read-only test of supplied batch, skips live merge. apply_existing_batch: requires confirm_live_merge=true to archive and write. The normal schedule is hourly (0 * * * *).

Main DAG dependency chain

start -> route_execution -> trigger_clinical_generator -> validate_clinical_files -> load_clinical_staging -> compare_clinical_incremental -> summarize_incremental_comparison -> merge_clinical_dry_run -> verify_merge_dry_run -> choose_merge_execution -> archive_bronze_before_merge -> merge_clinical_live -> process_clinical_silver -> end. Existing batch route joins at validate_clinical_files; skip_live_merge branches directly to end.

Airflow controls

The DAG uses PythonOperator, BranchPythonOperator, CloudRunExecuteJobOperator and EmptyOperator; retries=2, retry_delay=5 minutes, catchup=False, max_active_runs=1. TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS is used at branch joins and end. The DAG uses XCom for small comparison and dry-run totals; actual batch data lives in GCS/BigQuery rather than XCom. batch_id is derived from the logical run timestamp in normal mode.

Bronze ingestion and comparison

The generator writes batch-scoped CSVs. Validation confirms the expected 12 inputs. Staging tables receive the incoming batch. Comparison categorizes rows as new, changed or unchanged relative to Bronze. The dry-run MERGE totals are checked against the comparison totals before live changes are allowed.

Bronze backup, MERGE and audit

An archive is created before the live Bronze MERGE. MERGE inserts new keys and updates changed records. Audit table clinical_control.incremental_merge_audit records batch_id, table_name, new_records, changed_records, unchanged_records, status, execution_timestamp and error_message. A previously completed live batch succeeded for all 12 tables with 1,652 new, 3,858 changed and 49,595 unchanged.

Silver transformation details

bronze_to_silver.py reads target BigQuery Silver schemas dynamically, maps Bronze columns by name, trims strings, converts blanks to NULL, applies SAFE_CAST to typed fields, deduplicates source records by primary key, and performs BigQuery MERGE into each corresponding Silver table.

SQL operations are atomic per table; the overall 12-table loop is not one cross-table transaction. Bad casts can become NULL; stronger reject/error tracking is a recommended enhancement.

Incremental semantics and reruns

MERGE is intended to be idempotent for a stable source batch and unique keys, unlike blind append. Source deduplication reduces duplicate-key MERGE errors. A retry may reprocess earlier successful Silver tables; table-level MERGE makes that feasible. Deterministic tie-breaking does not automatically mean latest-event-wins without a trusted update timestamp.

Schema and physical design

Observed Silver types include STRING, DATE, NUMERIC, BOOL and FLOAT64. The schema is discovered from BigQuery metadata rather than maintained as a repeated hardcoded field list. Partition and clustering settings of the deployed tables were not verified, so no specific implemented partition key or byte-scan reduction is claimed.

Security and governance

Service accounts and GCP IAM provide the authentication/authorization model. The specific least-privilege role inventory, policy tags, row-access policies, column masking and CMEK configuration were not verified. For real patient information, restrict access, minimize PII, establish retention and auditing, and use approved governance controls.

Incident 1: Invalid FLOAT type

Incident: Silver processing succeeded for adverse_events, clinical_sites, clinical_studies and investigators, then failed while processing lab_results. Root cause: generated BigQuery SQL used SAFE_CAST(... AS FLOAT), but BigQuery expects FLOAT64. Resolution: corrected type mapping, uploaded bronze_to_silver.py to the Composer DAG bucket, reran the Silver task; logs subsequently showed all 12 table MERGEs succeeded. Lesson: normalize warehouse metadata type aliases before generating SQL.

Incident 2: Queued DAG runs

Incident: a manual run was queued while the team had triggered runs via both Airflow UI and SDK. Relevant conditions: the DAG had been paused and max_active_runs=1. Action: inspected run state and unpaused scheduling; the SDK-triggered run progressed to running. Do not assert worker saturation or XCom overload—those were not established root causes.

Incident 3: Read-only versus live execution

Incident/risk: testing incremental logic could accidentally modify Bronze. Action: introduced test_existing_batch and apply_existing_batch with explicit confirm_live_merge=true; a read-only run correctly skipped archive_bronze_before_merge and merge_clinical_live. Lesson: enforce destructive-operation controls through DAG branches, not only operator instructions.

Incident 4: Earlier PySpark practice issues

In a separate practice build, Dataproc Serverless jobs encountered missing PySpark config objects in GCS, a missing BigQuery dataset, and a Spark-BigQuery connector temporary bucket requirement. These are genuine development troubleshooting examples, but not incidents from the finished 12-table BigQuery SQL DAG.

CLI commands used

Upload DAG module: gcloud storage cp "C:\...\bronze_to_silver.py" gs://us-central1-paraxelairflow-aae43654-bucket/dags/bronze_to_silver.py

Trigger DAG: gcloud composer environments run paraxelairflow --location=us-central1 --project=vaarahigcpproject dags trigger -- clinical_data_pipeline --conf "{\"mode\":\"normal\"}"

Inspect tasks: gcloud composer environments run paraxelairflow --location=us-central1 --project=vaarahigcpproject tasks list -- clinical_data_pipeline --tree

Query BigQuery: bq query --use_legacy_sql=false "SELECT COUNT(*) FROM `vaarahigcpproject.paraxel_silver.patients` "

Remaining operational caveats

Latest screenshot showed process_clinical_silver success for all 12 tables. A separate final overall DAG run success-state capture was not provided. Gold materialized-view freshness and actual IAM configuration were not independently validated. Do not describe these as fully audited production guarantees.

Part 3 - Optimization and Interview Reference

Implementation-based reference | October 2026 | Synthetic clinical dataset

Evidence standard: observed implementation and logs are separated from unverified production claims.

Optimization already in design

Incremental comparison avoids treating every incoming row as a new insert. MERGE handles upserts. BigQuery Silver schema-driven conversions reduce duplicated hand-written code. Gold has two standard and two materialized views. Airflow max_active_runs=1 avoids concurrent runs of this DAG. Bronze archives improve recoverability.

Optimizations to evaluate, not claim

Evaluate partitioning by clinically relevant event or ingestion dates, and clustering by frequently filtered study_id/site_id/patient_id when supported by table size and query patterns. Use explicit column projections and filter pushdown. Monitor bytes processed, slot usage and materialized-view refresh cost. Benchmark changes before stating percentage savings. Dataproc autoscaling, salting, broadcast joins and Spark shuffle tuning are not evidenced in this BigQuery SQL pipeline.

Metric cheat sheet

12 related clinical source tables. Generator previously produced 53,500 rows per batch. Successful live merge observed: 1,652 new, 3,858 changed, 49,595 unchanged; Bronze 55,105 to 56,757 rows; 55,105 rows archived before merge. Silver task ultimately logged successful MERGE for all 12 tables. Gold: two standard views and two materialized views. No verified 500 GB/day, 45-minute SLA, 35% savings, 99.5% uptime or six-month reliability record.

SQL patterns worth discussing

CTEs: compute study-level patient and site counts before joining to clinical_studies. COUNT(DISTINCT): avoid duplicate patients/sites in reporting. COALESCE: show zero counts for studies with no matching enrollments. MERGE: update existing keys and insert missing keys. SAFE_CAST: handle malformed input types without query failure, while monitoring unexpected NULL conversions. ROW_NUMBER: deduplicate per primary key. QUALIFY: optional alternative syntax for filtering windowed results.

Gold SQL example

WITH patient_counts AS (SELECT study_id, COUNT(DISTINCT patient_id) AS total_patients FROM `vaarahigcpproject.paraxel_silver.patient_enrollments` WHERE enrollment_status IN ('active', 'completed') GROUP BY study_id), site_counts AS (SELECT study_id, COUNT(DISTINCT site_id) AS total_sites FROM `vaarahigcpproject.paraxel_silver.clinical_sites` GROUP BY study_id) SELECT s.study_id, s.study_title, COALESCE(p.total_patients,0) AS total_patients, COALESCE(c.total_sites,0) AS total_sites FROM `vaarahigcpproject.paraxel_silver.clinical_studies` s LEFT JOIN patient_counts p USING(study_id) LEFT JOIN site_counts c USING(study_id);

60-second architecture interview answer

I implemented an hourly GCP clinical pipeline with Cloud Composer coordinating Cloud Run, GCS and BigQuery. The generator writes 12 batch-scoped clinical feeds to GCS. Airflow validates and stages them, compares incremental changes, verifies a dry-run, archives Bronze and applies a guarded MERGE. The Silver stage uses target schema metadata for data typing, normalization, deduplication and MERGE. Gold standard/materialized views provide enrollment and treatment reporting. The pipeline includes retry logic, audit records and read-only test modes.

CAR: complex incremental MERGE

Context: clinical feeds contain both existing and new study records. Action: compared staging with Bronze, categorized new/changed/unchanged, verified dry-run totals, archived Bronze, then ran a guarded MERGE. Result: one observed batch had 1,652 inserts and 3,858 changes while 49,595 rows were unchanged, reducing unnecessary insert behavior and keeping an archive for recovery.

CAR: Silver failure resolution

Context: the Silver task stopped after four successful tables. Action: traced BigQuery BadRequest Type not found: FLOAT, corrected generated type mapping to FLOAT64, deployed the module, and reran the task. Result: logs reported successful MERGEs across all 12 Silver tables. Lesson: schema-driven SQL needs canonical BigQuery type names and task-level retry safety.

CAR: safe testing

Context: we needed to validate incremental behavior without changing Bronze. Action: separated test_existing_batch from apply_existing_batch, required confirm_live_merge=true for live manual execution, and routed read-only runs to skip_live_merge. Result: the test execution skipped live MERGE as designed, demonstrating a controlled release path.

Interview Q&A: Why GCS before BigQuery?

GCS provides durable landing, replayability and a clear separation between source delivery and warehouse processing. It also keeps source batch files available if downstream loading fails.

Interview Q&A: Why Composer rather than cron?

Composer manages a multi-stage DAG with branches, retries, task dependencies, cloud operators and observable task histories; cron alone would require custom dependency and recovery logic.

Interview Q&A: Why BigQuery MERGE?

MERGE combines conditional UPDATE and INSERT in a single table operation, supporting incremental upserts. Source uniqueness and deterministic matching remain important.

Interview Q&A: What is the Gold layer?

Business-facing BigQuery views and materialized views built on Silver. Existing examples summarize study enrollment, treatment-arm counts, event monitoring and enrollment status. The standard

views automatically query current Silver data; materialized views follow their configured refresh behavior.

Interview Q&A: What was not implemented?

Do not claim production Spark joins, Dataflow, Pub/Sub, source SFTP, 700-study production scale, star/snowflake marts, SCD Type 2, policy-tag masking, CMEK, verified partition/clustering, or quantified cost/latency improvements unless independently demonstrated for this exact project.

Final quick-reference checklist

Project vaarahigcpproject | Region us-central1 | Composer paraxelairflow | DAG clinical_data_pipeline | GCS personalproject_1 | Bronze paraxel_bronze | Silver paraxel_silver | Gold paraxel_gold | Audit clinical_control.incremental_merge_audit | 12 clinical tables | Hourly schedule | Branch modes normal, test_existing_batch, apply_existing_batch | Final observed Silver task success across 12 tables.